SIEM LAB SETUP GUIDE

Wazuh · DVWA · Splunk — Week 6 Companion Guide

TechRise 3.0 · Cybersecurity & DevSecOps · Cohort 3.0

Installation · Integration · Testing · PayliteNG Detection Labs

Student Name: _____

ARCHITECTURE OVERVIEW

■ PayliteNG Context

PayliteNG's breach was undetected for three weeks because no monitoring tools existed. This lab builds the detection stack that would have caught @gr4yh4t within 60 seconds of the first attack.

Data Flow — How the Three Tools Connect

- DVWA Apache logs → bind-mounted to host → Splunk monitors host path → detects web attacks
- Wazuh agent (on Kali) → monitors host files/users/processes → sends to Wazuh manager (Docker)
- Wazuh manager → writes alerts.json → bind-mounted to host → Splunk ingests as index=wazuh
- Result: Splunk shows BOTH web attacks (from DVWA) AND host events (from Wazuh) in one search

What Each Tool Detects

- DVWA + Splunk: SQL injection, XSS, brute force, directory traversal — network/web layer
- Wazuh: file modifications, new users, cron changes, rootkits — host/OS layer
- Splunk + Wazuh combined: complete attack chain from first web request to backdoor installation

PART 1 Install Docker

Required for both DVWA and Wazuh Manager

BASH

```
sudo apt update && sudo apt install docker.io docker-compose-v2 -y sudo systemctl start docker &&
sudo systemctl enable docker sudo usermod -aG docker $USER # Log out and back in for group change
to apply — IMPORTANT # Verify: docker --version docker-compose version
```

✓ Expected Output: Docker ready

```
Docker version 24.x Docker Compose version v2.x
```

✓ Verification Checklist — Tick When Done

- ■ Docker installed and running
- ■ Current user added to docker group
- ■ Logged out and back in

PART 2 Deploy Wazuh Manager, Indexer, and Dashboard

Runs in Docker — v4.7.3

■ Use exactly v4.7.3

Do not use main or latest. Version drift between manager and dashboard causes hard-to-diagnose errors.

Step 1 — Clone the repository at the exact version:

BASH

```
mkdir -p ~/labs/week6 cd ~/labs/week6 git clone https://github.com/wazuh/wazuh-docker.git -b v4.7.3
cd wazuh-docker/single-node
```

Step 2 — Set kernel parameter required by the OpenSearch indexer:

BASH

```
sudo sysctl -w vm.max_map_count=262144 echo 'vm.max_map_count=262144' | sudo tee -a
/etc/sysctl.conf
```

Step 3 — Generate SSL certificates:

BASH

```
docker compose -f generate-indexer-certs.yml run --rm generator
```

■ Verify certificates were created as FILES not directories: ls -lh config/wazuh_indexer_ssl_certs/ — every .pem file must have a size greater than 0 bytes

Step 4 — Start the Wazuh stack:

BASH

```
docker compose up -d docker compose ps
```

✓ Expected Output: All three containers running

```
NAME STATUS single-node-wazuh.manager-1 Up single-node-wazuh.indexer-1 Up single-node-wazuh.dashboard-1 Up
```

Step 5 — Access the Wazuh Dashboard:

Dashboard Access

- URL: https://localhost (accept the self-signed certificate warning)
- Username: admin
- Password: SecretPassword
- If you see 'server is not ready yet' — wait 1-2 minutes and hard-refresh (Ctrl+Shift+R)
- If you see 'Error fetching data' — generate a relevant event first (covered in testing section)

✓ Part 2 Complete — Wazuh Manager Running

- ■ All three containers show Up in docker compose ps
- ■ https://localhost opens the Wazuh dashboard
- ■ Can log in with admin / SecretPassword

PART 3 Install Wazuh Agent Natively on Kali

Native install — NOT Docker

Why Native Agent (Not Docker Agent)

- The official wazuh/wazuh-agent Docker image has a known bug — it fails to write WAZUH_MANAGER into its config, causing 'Invalid server address found: ' errors
- wazuh-agent and wazuh-manager CANNOT both be installed natively on the same host — packages conflict
- Since our manager runs in Docker (separate filesystem), the native agent works perfectly alongside it
- This also matches production style: manager on one server, agents on the hosts being monitored

Step 1 — Add Wazuh repository:

BASH

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | \ sudo gpg --dearmor -o
/usr/share/keyrings/wazuh.gpg echo 'deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt/ stable main' | \ sudo tee /etc/apt/sources.list.d/wazuh.list
sudo apt update
```

Step 2 — Install matching version (must match manager v4.7.3):

BASH

```
sudo apt install wazuh-agent=4.7.3-1
```

Step 3 — Point agent at the manager (Docker exposes port 1514/1515 to localhost):

BASH

```
sudo sed -i 's/MANAGER_IP/localhost/g' /var/ossec/etc/ossec.conf # Verify the change sudo grep -A3
'' /var/ossec/etc/ossec.conf
```

✓ Expected Output: Manager address set correctly

```
localhost
```

Step 4 — Start and enable the agent:

BASH

```
sudo systemctl daemon-reload sudo systemctl enable wazuh-agent sudo systemctl start wazuh-agent #
Verify sudo systemctl status wazuh-agent | grep Active
```

✓ Expected Output: Agent running

```
Active: active (running)
```

Step 5 — Verify connection from the manager side:

BASH

```
docker exec -it single-node-wazuh.manager-1 \ /var/ossec/bin/agent_control -l
```

✓ Expected Output: Agent actively connected

```
Available agents: ID: 001, Name: kali, IP: any Active ← Active confirms live communication — not just registered
```

■ **Also check the Wazuh dashboard → Agents tab — your host should appear with a green Active status**

✓ Part 3 Complete — Wazuh Agent Connected

- ■ wazuh-agent=4.7.3-1 installed
- ■ ossec.conf shows localhost as server address
- ■ sudo systemctl status wazuh-agent shows active (running)
- ■ docker exec agent_control -l shows agent as Active
- ■ Wazuh dashboard → Agents shows green Active status

PART 4 Enable Real-Time File Integrity Monitoring

Detect file changes within seconds

By default Wazuh scans monitored directories every 12 hours. For lab demos you need real-time detection using inotify so changes appear within seconds.

Step 1 — Check current syscheck configuration:

BASH

```
sudo grep -A5 '' /var/ossec/etc/ossec.conf
```

Step 2 — Enable realtime monitoring on key directories:

BASH

```
sudo sed -i 's|/etc,/usr/bin,/usr/sbin|/etc,/usr/bin,/usr/sbin|' \ /var/ossec/etc/ossec.conf #
Restart agent to apply change sudo systemctl restart wazuh-agent
```

Step 3 — Test FIM is working:

BASH

```
# Modify a monitored file sudo bash -c "echo '# fim test' >> /etc/passwd" # Watch agent log for the
event sudo tail -n 15 /var/ossec/logs/ossec.log # Confirm alert reached the manager docker exec -it
single-node-wazuh.manager-1 \ grep "File '/etc/passwd' modified" /var/ossec/logs/alerts/alerts.log
# Clean up sudo sed -i '/# fim test/d' /etc/passwd
```

✓ Expected Output: FIM working

```
File '/etc/passwd' modified Rule: 550 — Integrity checksum changed ← Alert appears within seconds of the
file change
```

■ **Check the dashboard: Modules → File Integrity Monitoring — the /etc/passwd event appears there**

✓ Part 4 Complete — Real-Time FIM Active

- ■ realtime=yes added to syscheck directories in ossec.conf
- ■ Agent restarted after config change
- ■ FIM test — /etc/passwd modification — triggered an alert in manager
- ■ Alert visible in Wazuh dashboard → File Integrity Monitoring

PART 5 — Set Up DVWA with Apache Logs Mounted to Host

Attack target + log visibility

■ PayliteNG Context

DVWA is the replica of PayliteNG's customer portal. Every SQL injection, XSS, and brute force attack you run against DVWA generates Apache log entries — the same type @gr4yh4t generated against PayliteNG. Mounting the logs to the host lets Splunk read them directly.

Step 1 — Create the log directory and run DVWA with bind-mount:

BASH

```
mkdir -p ~/labs/week6/dvwa_logs docker run -d --name dvwa \ -p 80:80 \ -v
~/labs/week6/dvwa_logs:/var/log/apache2 \ vulnerables/web-dvwa
```

■ The -v flag mounts the container's Apache log directory to your Kali filesystem. Splunk reads from ~/labs/week6/dvwa_logs/ — no need to exec into the container.

Step 2 — Verify DVWA is running and logs are accessible:

BASH

```
docker ps | grep dvwa curl -s http://localhost/ -o /dev/null -w "%{http_code}\n" # Generate some
traffic to create the log file curl -s http://localhost/login.php -o /dev/null curl -s
http://localhost/ -o /dev/null # Confirm log file exists on host ls -lh ~/labs/week6/dvwa_logs/ cat
~/labs/week6/dvwa_logs/access.log | tail -5
```

✓ Expected Output: DVWA running and logs visible on host

```
200 ← DVWA responding dvwa_logs/ access.log ← Apache log accessible on host filesystem 127.0.0.1 - -
[20/Jan/2026] "GET /login.php HTTP/1.1" 200 3421
```

Step 3 — Complete DVWA first-time setup in browser:

Browser Setup Steps

- Open Firefox → go to http://localhost
- Login: admin / password
- Click: Create / Reset Database (bottom of the page)
- Wait for success message then log in again
- Go to: DVWA Security → Set to Low → Submit
- DVWA is now ready for all Week 6 labs

■ If DVWA shows Exited in docker ps

Run: docker start dvwa — this happens after a reboot or if the container was stopped to save RAM

✓ Part 5 Complete — DVWA Ready

- ■ DVWA container running — docker ps shows Up
- ■ http://localhost opens DVWA login page
- ■ Database created and can log in as admin/password
- ■ Security level set to Low
- ■ ~/labs/week6/dvwa_logs/access.log exists and has content

PART 6 Install and Configure Splunk

SIEM — ingests DVWA and Wazuh data

Step 1 — Download and install Splunk:

BASH

```
# Download from splunk.com (register first — free account) # File: splunk-9.x.x-linux-amd64.deb
sudo dpkg -i ~/Downloads/splunk*.deb sudo /opt/splunk/bin/splunk start --accept-license # Set admin
username and password when prompted sudo /opt/splunk/bin/splunk enable boot-start
```

Splunk Access

- URL: http://localhost:8000
- Username: admin
- Password: whatever you set during installation
- If you forgot the password: sudo rm /opt/splunk/etc/passwd then restart Splunk

Step 2 — Create the DVWA index:

SPLUNK UI

```
# In Splunk Web UI: # Settings → Indexes → New Index # Index Name: dvwa # Max Size: 500 MB # Save
```

Step 3 — Add DVWA Apache logs as data source:

SPLUNK UI

```
# Settings → Data Inputs → Files & Directories → Add New # File:
/home/kali/labs/week6/dvwa_logs/access.log # Source type: access_combined # Index: dvwa # Save
```

Step 4 — Verify DVWA logs are ingesting:

SPLUNK SEARCH BAR

```
index=dvwa | head 10
```

✓ Expected Output: DVWA logs in Splunk

```
Events appear showing Apache log entries from DVWA Each line shows: clientip, method, uri, status, bytes
```

✓ Part 6 Complete — Splunk Ingesting DVWA Logs

- ■ Splunk accessible at http://localhost:8000
- ■ dvwa index created
- ■ DVWA access.log added as data source
- ■ index=dvwa | head 10 returns events

PART 7 Connect Wazuh Alerts to Splunk

The critical integration step

■ PayliteNG Context

This is the step that unifies both tools. Once complete — Splunk shows DVWA web attacks (index=dvwa) AND Wazuh host-based detections (index=wazuh) in one place. A single Splunk search covers the full PayliteNG attack chain.

Step 1 — Add a volume mount to expose Wazuh alerts to the host:

Edit the docker-compose.yml file:

BASH

```
cd ~/labs/week6/wazuh-docker/single-node nano docker-compose.yml
```

Find the wazuh.manager service volumes section and add this line:

docker-compose.yml — add this line

```
# Add this under the existing volumes: list for wazuh.manager -
~/labs/week6/wazuh_alerts:/var/ossec/logs/alerts
```

Step 2 — Create the alerts directory and recreate the manager:

BASH

```
mkdir -p ~/labs/week6/wazuh_alerts docker compose up -d --force-recreate wazuh.manager # Wait 30
seconds for manager to restart sleep 30 # Verify the alerts directory is being written to ls -la
~/labs/week6/wazuh_alerts/
```

✓ Expected Output: Wazuh alerts accessible on host

```
alerts.json ← Wazuh writing alerts directly to host filesystem alerts.log ← Plain text version
```

Step 3 — Create the Wazuh index in Splunk:

SPLUNK UI

```
# Settings → Indexes → New Index # Index Name: wazuh # Max Size: 500 MB # Save
```

Step 4 — Add Wazuh alerts as a Splunk data source:

SPLUNK UI

```
# Settings → Data Inputs → Files & Directories → Add New # File:
/home/kali/labs/week6/wazuh_alerts/alerts.json # Source type: _json # Index: wazuh # Save
```

Step 5 — Generate a test event and verify in Splunk:

BASH + SPLUNK SEARCH

```
# Generate a Wazuh alert (file modification) sudo bash -c "echo '# splunk integration test' >> /etc/hosts" sleep 10 # Search in Splunk # index=wazuh | head 10 # Clean up sudo sed -i '/splunk integration test/d' /etc/hosts
```

✓ Expected Output: Wazuh alerts in Splunk

Events appear in Splunk showing Wazuh JSON alerts Fields visible: rule.id, rule.description, agent.name, timestamp

Step 6 — Verify the unified search works:

SPLUNK SEARCH BAR — THE UNIFIED VIEW

```
# This single search shows BOTH DVWA web attacks AND Wazuh host events index=dvwa OR index=wazuh | table _time index source _raw | sort -_time
```

✓ Expected Output: Unified SOC dashboard working

Events from both dvwa and wazuh indexes appear together Sorted by time — shows the full attack timeline in one view

✓ Part 7 Complete — Wazuh and Splunk Connected

- ■ Wazuh alerts volume mounted in docker-compose.yml
- ■ ~/labs/week6/wazuh_alerts/alerts.json exists and has content
- ■ wazuh index created in Splunk
- ■ Wazuh alerts.json added as Splunk data source
- ■ index=wazuh | head 10 returns Wazuh alert events
- ■ index=dvwa OR index=wazuh returns events from both tools

PART 8 — Testing — Prove All Three Tools Are Connected

Run these tests in order

■ PayliteNG Context

These tests recreate the PayliteNG attack chain. Every attack you run should appear in Splunk within 30 seconds. Host-based attacks should appear in Wazuh alerts. Run all six tests before the Checkpoint Project.

Setup — Open four windows before starting:

SETUP — DO THIS FIRST

```
# Window 1 — Splunk (in browser) # http://localhost:8000 # Search: index=dvwa OR index=wazuh | table _time index _raw # Set auto-refresh: Every 30 seconds # Window 2 — Wazuh dashboard (in browser) # https://localhost # Modules → Security Events # Window 3 — Wazuh alerts live (terminal) docker exec single-node-wazuh.manager-1 \ tail -f /var/ossec/logs/alerts/alerts.log # Window 4 — Attack terminal # Use this for running all test commands below
```

TEST 1 — SQL Injection Against DVWA

Detected by: Splunk

In Firefox — DVWA → SQL Injection — type this and click Submit:

DVWA BROWSER INPUT

```
1' UNION SELECT user,password FROM users--
```

In Splunk search bar:

SPLUNK SPL — RULE 4

```
index=dvwa sourcetype=access_combined | rex field=uri_path "(?i)(?Punion|select|'|--)" | where isnotnull(sqli) | table _time, clientip, uri_path, sqli, status
```

Splunk: ■ SQL injection request appears within 30 seconds. uri_path shows the UNION SELECT payload.

Wazuh: ■ No alert — this is a network-level attack, not a host change.

Connection to PayliteNG:

- This is exactly what 203.0.113.88 sent to PayliteNG: GET /login.php?id=1+UNION+SELECT+user,password+FROM+users--
- If Splunk had been running on January 13th — this would have alerted within seconds

TEST 2 — Brute Force Attack on DVWA

Detected by: Splunk

In Firefox — DVWA → Brute Force. Run Burp Suite Intruder with your wordlist. Then:

SPLUNK SPL — RULE 1 (web brute force)

```
index=dvwa sourcetype=access_combined uri="*brute*" | stats count by clientip | where count > 10 | sort -count
```

Splunk: ■ Request count climbs. Rule fires when threshold exceeded.

Wazuh: ■ No host-level alert — brute force is a network event.

TEST 3 — SSH Brute Force (Both Tools)

Detected by: Both

BASH — Window 4

```
# Window 4 — generate SSH brute force for i in {1..15}; do ssh -o StrictHostKeyChecking=no \ -o ConnectTimeout=1 \ wronguser@localhost 2>/dev/null sleep 0.5 done
```

SPLUNK SPL — RULE 1

```
# Splunk SPL — check auth.log (if ingested) or use practice_auth.log index=payliteng "Failed password" | rex "from (?P\d+\.\d+\.\d+\.\d+)" | stats count by src_ip | where count > 10
```

Splunk: ■ Failed password events counted by IP. Rule fires above threshold.

Wazuh: ■ Authentication failure alerts appear in Window 3 and dashboard.

This is the only test where BOTH tools fire on the same event. SSH brute force appears in auth.log (Splunk) AND triggers Wazuh authentication rules (Wazuh).

TEST 4 — File Modification — Backdoor Simulation

Detected by: Wazuh

■ PayliteNG Context

In Week 4 @gr4yh4t installed a cron backdoor on PayliteNG's server. This test proves Wazuh catches that — while Splunk completely misses it.

BASH — Window 4

```
# Simulate attacker modifying system file sudo bash -c "echo '# attacker modified this' >> /etc/hosts" # Watch Window 3 — Wazuh alert fires within seconds # Then clean up sudo sed -i '/attacker modified this/d' /etc/hosts
```

Wazuh: ■ Rule 550 fires: Integrity checksum changed for /etc/hosts. Appears in Window 3 and Splunk index=wazuh

Splunk (dvwa index): ■ No alert — file changes do not appear in web logs

SPLUNK SPL — unified view catches it

```
# But Splunk DOES see it via Wazuh integration: index=wazuh | search "/etc/hosts" | table _time rule.description agent.name
```

Teaching point: Splunk alone missed this. Wazuh caught it. Together — index=wazuh in Splunk shows the event.

TEST 5 — Cron Backdoor Installation

Detected by: Wazuh

■ PayliteNG Context

This is the exact attack from Week 4 — @gr4yh4t installed a cron job that ran every 5 minutes. Splunk had no way to detect it. Wazuh catches it immediately.

BASH — Window 4

```
# Simulate the Week 4 backdoor echo '*/5 * * * * root /tmp/phone_home.sh' | \ sudo tee -a /etc/crontab # Watch Window 3 — crontab modification alert fires sleep 10 # Clean up sudo sed -i '/phone_home/d' /etc/crontab
```

Wazuh: ■ /etc/crontab modified alert fires. Level 8 severity. Visible in dashboard and index=wazuh in Splunk.

Splunk (dvwa index): ■ Completely missed — crontab changes are not in web or FTP logs

TEST 6 — New User Creation — Backdoor Account

Detected by: Wazuh

BASH — Window 4

```
# Simulate attacker creating a backdoor user sudo useradd backdoor_account 2>/dev/null # Watch Window 3 for the alert sleep 5 # Clean up sudo userdel backdoor_account
```

Wazuh: ■ Rule 5905: New user added to the system. Level 8 severity. Immediate alert.

Splunk (dvwa index): ■ Completely missed — user creation not in Apache logs

TEST RESULTS SUMMARY

The Key Teaching Point

- Splunk (dvwa index) alone: catches web attacks, misses host-based attacks
- Wazuh alone: catches host changes, misses web-layer attacks
- Splunk (wazuh index): bridges the gap — host events searchable in Splunk
- All three together: complete visibility across the entire attack chain
- This is Defence in Depth — no single tool catches everything

COMMON ISSUES AND FIXES

FINAL CHECKLIST — COMPLETE SETUP

✓ All items complete — ready for Checkpoint Project

- ■ PART 1: Docker installed — docker --version and docker-compose version both work
- ■ PART 2: Wazuh manager, indexer, dashboard all show Up in docker compose ps
- ■ PART 2: Wazuh dashboard accessible at https://localhost — can log in
- ■ PART 3: Wazuh agent installed natively at version 4.7.3-1
- ■ PART 3: Agent shows Active in docker exec agent_control -l
- ■ PART 3: Wazuh dashboard → Agents shows green Active status
- ■ PART 4: realtime=yes enabled in syscheck config
- ■ PART 4: FIM test — /etc/passwd modification generated an alert
- ■ PART 5: DVWA running with logs bind-mounted to ~/labs/week6/dvwa_logs/
- ■ PART 5: DVWA accessible at http://localhost — security level Low
- ■ PART 6: Splunk accessible at http://localhost:8000
- ■ PART 6: index=dvwa | head 10 returns Apache log events
- ■ PART 7: Wazuh alerts volume mounted in docker-compose.yml
- ■ PART 7: ~/labs/week6/wazuh_alerts/alerts.json exists and has content
- ■ PART 7: index=wazuh | head 10 returns Wazuh JSON alert events
- ■ PART 7: index=dvwa OR index=wazuh returns events from BOTH tools
- ■ TEST 1: SQL injection in DVWA → appears in Splunk within 30 seconds
- ■ TEST 3: SSH brute force → appears in BOTH Splunk and Wazuh
- ■ TEST 4: File modification → caught by Wazuh, visible in Splunk index=wazuh
- ■ TEST 5: Cron backdoor → caught by Wazuh only — Splunk dvwa index missed it
- ■ TEST 6: New user → caught by Wazuh only — proves why both tools are needed